EE3005 IMAGE PROCESSING

Assignment 1

Fundamentals of Image Representation and Enhancement

Student	Burak Bekir Çakırer
Student ID	2211011008
Course	EE3005 Image Processing
Submission Date	_____

MATLAB implementation, visual results and technical analysis

Contents

1. Introduction
2. Task 1 - Image Reading and Matrix Representation
3. Task 2 - Separating RGB Channels
4. Task 3 - Summary and Applications
5. Task 4 - DICOM Reading and Metadata Extraction
6. Task 5 - Point Operations
7. Task 6 - Histogram Analysis
8. Task 7 - Histogram Equalization
9. Task 8 - Image Enhancement Comparison
10. Implementation Challenges and Notes
11. Conclusion

Appendix A - Complete MATLAB Source Code

The report follows the same order as the assignment rubric. All images used are from the supplied Image Samples folder, so the analysis is reproducible from the repository.

1. Introduction

The purpose of this assignment is to practice the basic representation and enhancement operations used in digital image processing. I used the supplied chest X-ray image as the main grayscale test image, the supplied pseudo-color brain image for RGB operations, and image4.dcm for the DICOM part. The implementation was written in MATLAB using matrix indexing and functions from the Image Processing Toolbox.

The work is organized around eight tasks: reading an image as a numerical matrix, separating RGB planes, explaining applications, reading DICOM data, applying point transformations, analyzing histograms, performing global histogram equalization, and comparing enhancement methods. All processing is kept on an 8-bit 0-255 scale for the standard images, while the original DICOM pixel data is preserved and scaled only for visualization.

2. Task 1 - Image Reading and Matrix Representation

2.1 Image reading

I selected image3.jpg, a chest radiograph. The file is stored as a three-channel JPEG, but visually it is grayscale. Therefore, after imread I check the third dimension and convert the image with rgb2gray when necessary. The resulting matrix has 752 rows and 800 columns.

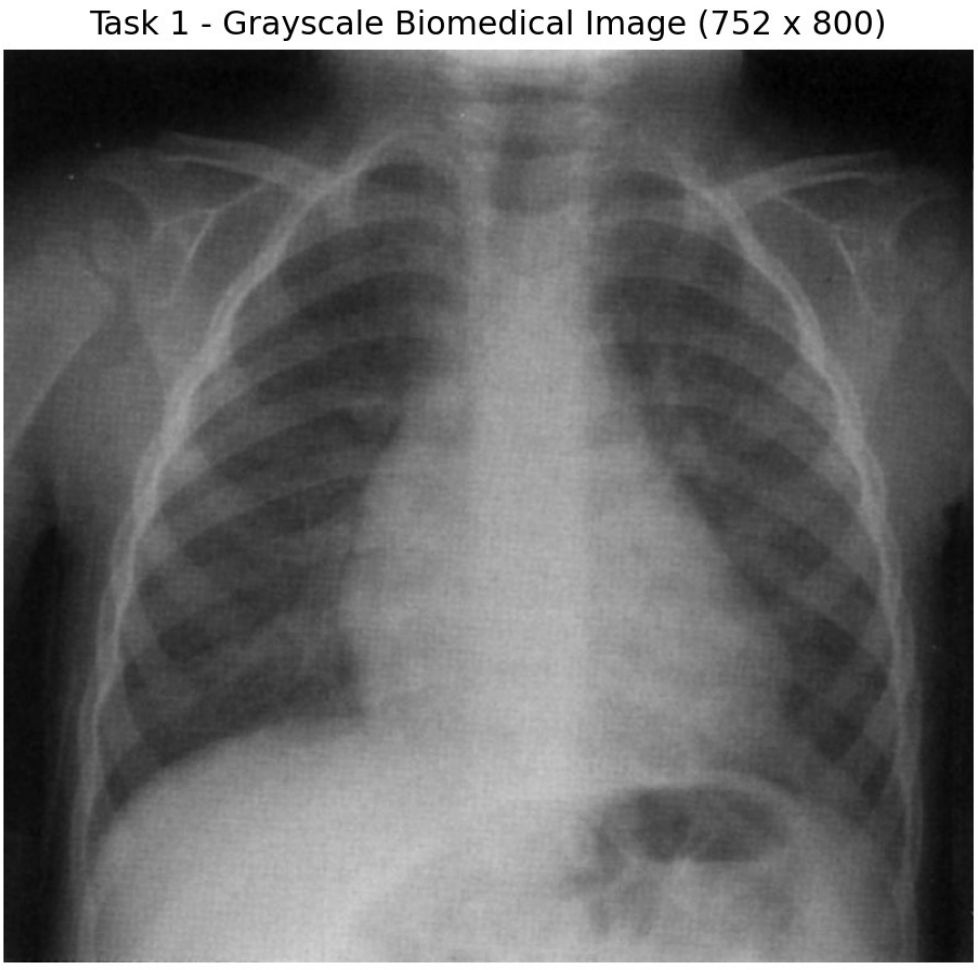


Figure 1. Grayscale chest X-ray used for the matrix and enhancement experiments.

2.2 Matrix interpretation

A grayscale image can be treated as a two-dimensional matrix I(r,c). Each row-column location corresponds to one pixel. For an 8-bit image, a matrix element is an integer between 0 and 255: 0 represents black, 255 represents white, and intermediate values represent gray levels. This representation is important because most image processing operations are numerical operations on these matrix elements.

$$\text{Grayscale pixel: } I(r,c) \in \{0,1,\ldots,255\}$$

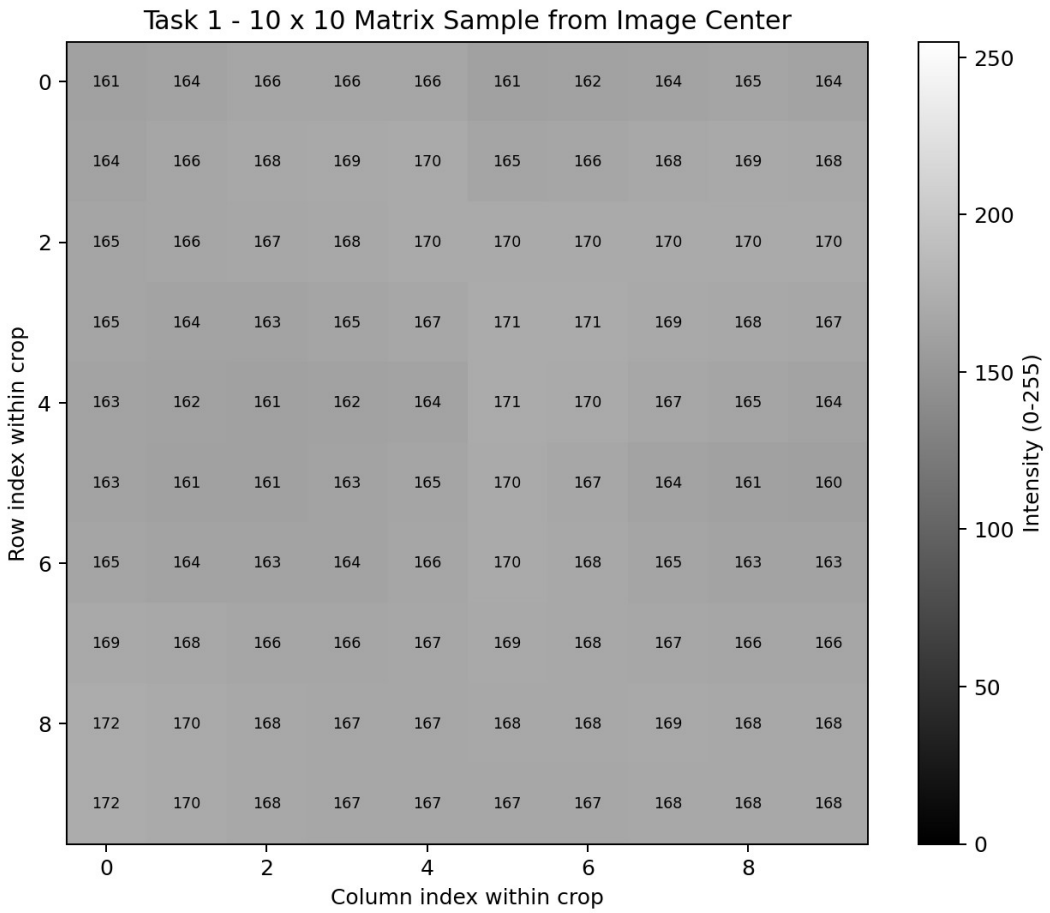


Figure 2. A 10 x 10 sample from the center of the image shown as actual matrix values.

The sample above contains values mostly around 160-172. This region is relatively bright compared with the darker lung regions. The function displayMatrixRepresentation.m prints the same block in the MATLAB Command Window and visualizes it with imagesc.

2.3 MATLAB code used for Task 1

```
img1 = imread(fullfile(dataDir, 'image3.jpg'));
if size(img1,3) == 3
    grayImg = rgb2gray(img1);
else
    grayImg = img1;
end

displayMatrixRepresentation(grayImg, outDir);
```

3. Task 2 - Separating RGB Channels

3.1 RGB representation

An RGB image is stored as an M x N x 3 array. For each pixel, the first plane contains red intensity, the second plane contains green intensity, and the third plane contains blue intensity. The displayed color is produced by the combination [R,G,B]. Channel separation therefore only requires indexing the third dimension of the array.

$$RGB\ pixel:\ I(r,c,:) = [\ R(r,c),\ G(r,c),\ B(r,c)\ ]$$

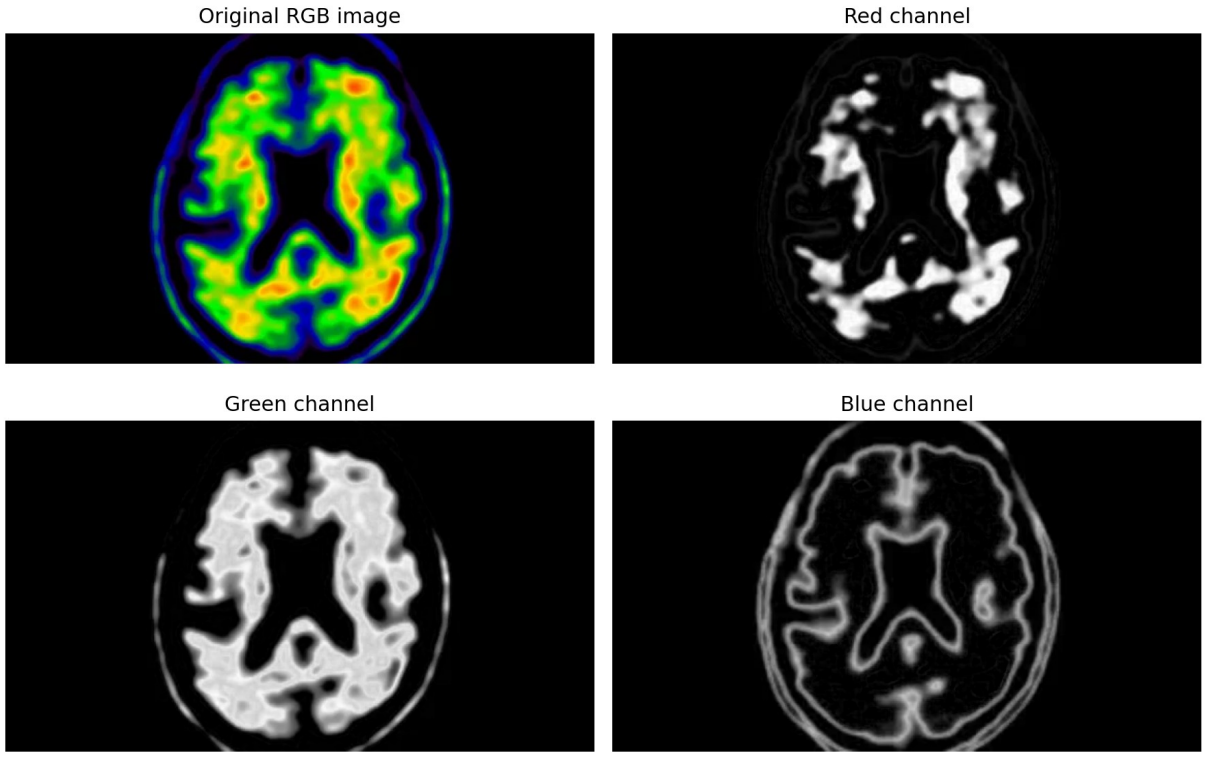


Figure 3. Original pseudo-color biomedical image and the separated R, G and B channels.

3.2 Observation

The green channel is visually the strongest over large parts of the tissue. Its mean intensity is 41.87, compared with 20.02 for red and 19.05 for blue. The red channel emphasizes the yellow/red high-intensity regions, while the blue channel mainly highlights the outer blue structures and boundaries. The same anatomical structure is present in all three matrices, but with different weights because the original image uses a false-color map.

Channel	Mean	Std. dev.	Median	95th percentile
Red	20.019	56.030	0	185
Green	41.867	79.144	0	222
Blue	19.048	40.818	0	127

Table 1. Basic statistics of the separated RGB channels.

3.3 MATLAB code used for Task 2

```
rgbImg = imread(fullfile(dataDir, 'image_rgb_matlab.png'));
R = rgbImg(:,:,1);
G = rgbImg(:,:,2);
B = rgbImg(:,:,3);

figure;
```

```
tiledlayout(2,2);
nexttile; imshow(rgbImg); title('Original RGB image');
nexttile; imshow(R); title('Red channel');
nexttile; imshow(G); title('Green channel');
nexttile; imshow(B); title('Blue channel');
```

4. Task 3 - Summary and Applications

Understanding the matrix representation is a foundation for almost every later image-processing method. Filtering, edge detection, segmentation, morphology and feature extraction all read or modify pixel values based on their positions in a matrix. It also makes it possible to reason about memory size, intensity range, neighborhood operations and geometric coordinates.

RGB separation is useful when color itself carries information. A system can process a single color plane if a feature is clearer there, or compare channels to build color-based rules. This is commonly useful in computer vision for object or surface detection, in medical or biological imaging for stain/marker analysis and false-color visualizations, and in multimedia processing for color correction, compression and enhancement.

- Computer vision: isolate objects using color differences, then perform segmentation or tracking on the most informative channel.
- Medical imaging: inspect false-color maps or multi-channel microscopy data and process channels independently before combining results.
- Multimedia: change color balance, white balance, saturation or channel gains without changing the image geometry.
- Pre-processing: convert matrices to normalized forms that can be used by classical algorithms or machine-learning models.

5. Task 4 - DICOM Reading and Metadata Extraction

5.1 Why DICOM is different

DICOM (Digital Imaging and Communications in Medicine) is more than a normal image file. A JPEG or PNG mainly stores pixel data and basic file information, while a DICOM object can store the pixel matrix together with clinical and acquisition metadata such as modality, scanner model, study date, slice thickness, field strength, repetition time and echo time. This makes DICOM suitable for hospital imaging workflows and for keeping acquisition context with the image.

Task 4 - DICOM MR Image (windowed display)

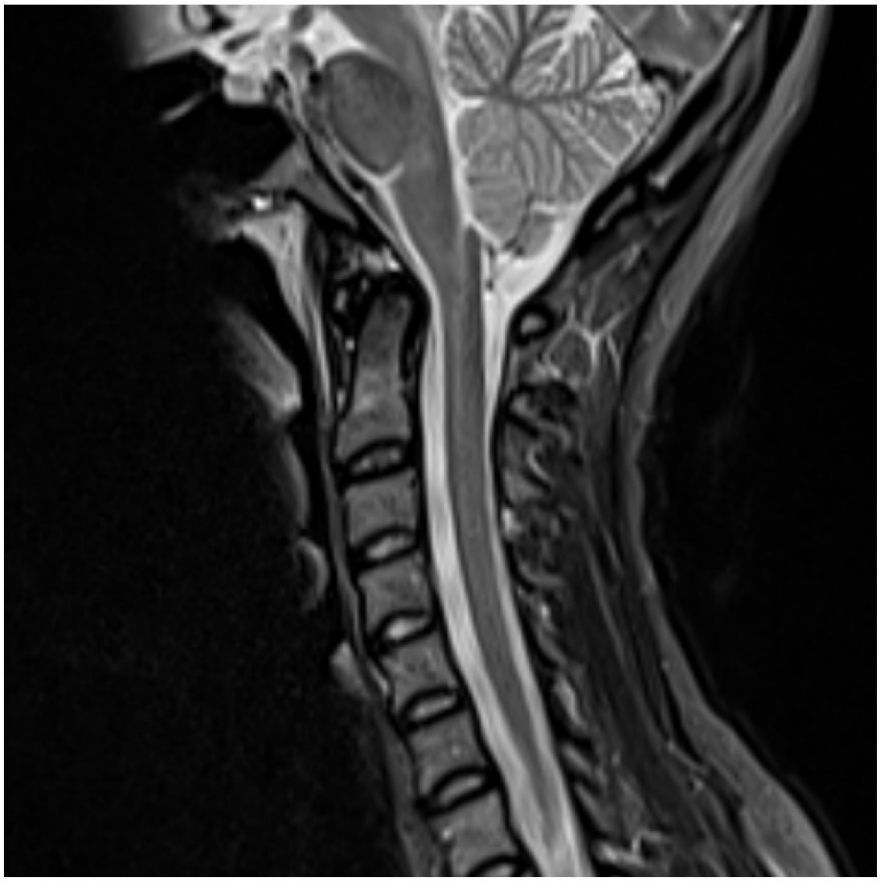


Figure 4. Supplied MR DICOM image displayed using its stored window information.

5.2 Extracted metadata

Metadata field	Extracted value
Patient name / label	DeepResolve_Altea1
Patient ID / label	2021.01.13-10:04:24-STD-1.3.12.2.1107.5.99.3
Birth date	19860706
Sex	F
Age	034Y
Modality	MR
Manufacturer	Siemens Healthineers
Scanner model	MAGNETOM Altea
Study date	20210113
Series description	Exported MR Basic Reading
Rows x Columns	512 x 512
Bits allocated	16
Photometric interpretation	MONOCHROME2
Slice thickness	2.5 mm
Repetition time (TR)	4460 ms
Echo time (TE)	52 ms

Magnetic field strength	1.5 T
Flip angle	150°
Window center / width	229.0 / 533.0

Table 2. Selected metadata extracted from image4.dcm. The supplied sample uses technical/de-identified labels rather than a normal patient name.

5.3 DICOM-specific considerations

The DICOM pixel matrix is 512 x 512 and uses 16 allocated bits with 12 meaningful stored bits. Therefore, it should not be assumed to use the same 0-255 range as an 8-bit JPEG. For display, MATLAB can scale the available range with imshow(dicomImg,[]) or use the WindowCenter and WindowWidth fields. The stored data should not be converted to uint8 before analysis unless there is a specific reason, because that would discard intensity precision.

5.4 MATLAB code used for Task 4

```
dicomPath = fullfile(dataDir, 'image4.dcm');
dicomImg = dicomread(dicomPath);
dicomMeta = dicominfo(dicomPath);

figure;
imshow(dicomImg, []);
title('DICOM MR Image');

displayDicomMetadata(dicomMeta);
```

6. Task 5 - Point Operations

A point operation calculates each output pixel only from the corresponding input pixel. I used the same chest X-ray so that the effect of the transformations can be compared directly. All results were clipped to the valid 8-bit interval [0,255].

Brightness: $g = clip(f + b)$ / Contrast: $g = clip(a \cdot f)$ / Negative: $g = 255 - f$

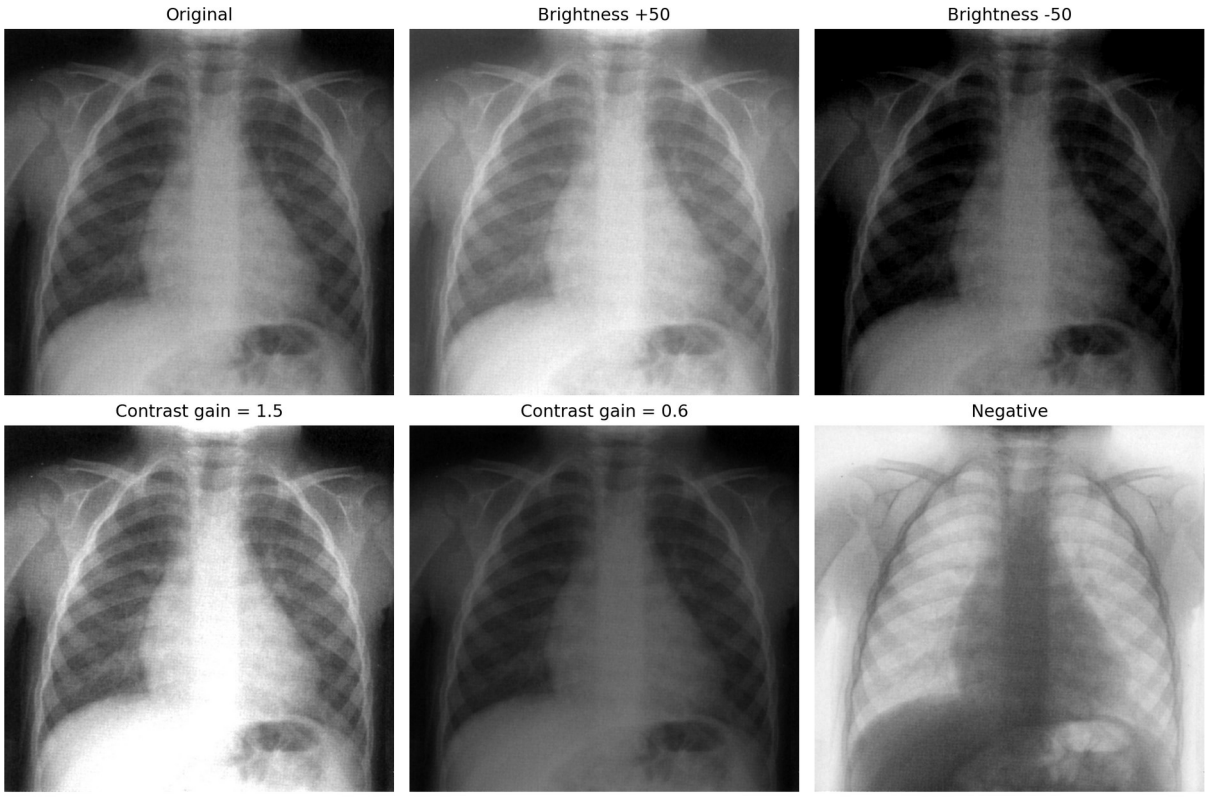


Figure 5. Original image, brightness changes, contrast changes and image negative.

6.1 Brightness

Adding +50 shifts most intensities to the right, so the complete image becomes lighter. The mean increases from 96.25 to 146.25. Subtracting 50 moves the distribution toward black. Low values are clipped to zero, which explains why the standard deviation also decreases and some shadow detail is lost.

6.2 Contrast

A gain larger than 1 spreads the values away from zero and produces stronger brightness differences. With gain 1.5, the standard deviation increases from 48.65 to 70.36, but bright regions can saturate at 255. A gain of 0.6 compresses the range; the image becomes darker and flatter, and the standard deviation falls to 29.19.

6.3 Negative

The negative transformation reverses the intensity scale. Dark pixels become bright and bright pixels become dark. The order of the spatial structures is unchanged, and the standard deviation remains the same because the transformation is a reflection around the middle of the range.

Result	Min	Max	Mean	Std. dev.
Original	0	212	96.247	48.646
Brightness +50	50	255	146.247	48.645
Brightness -50	0	162	50.390	42.759
Contrast x1.5	0	255	142.605	70.358
Contrast x0.6	0	127	57.349	29.188
Negative	43	255	158.753	48.646

Table 3. Numerical effect of the point transformations.

6.4 MATLAB code used for Task 5

```
bright       = clipUint8(double(grayImg) + 50);
dark         = clipUint8(double(grayImg) - 50);
highContrast = clipUint8(1.5 * double(grayImg));
lowContrast  = clipUint8(0.6 * double(grayImg));
negative     = 255 - grayImg;
```

7. Task 6 - Histogram Analysis

7.1 Grayscale histogram

A grayscale histogram counts how many pixels have each intensity. The x-axis represents the gray level from 0 to 255, and the y-axis is the number of pixels at that level. For this chest X-ray, most values lie in the dark-to-intermediate range, with strong groups roughly between 50 and 190. The 50th percentile is about 90 and the 95th percentile is about 179. This indicates that the image does not use the full bright end of the 8-bit range; its maximum is 212.

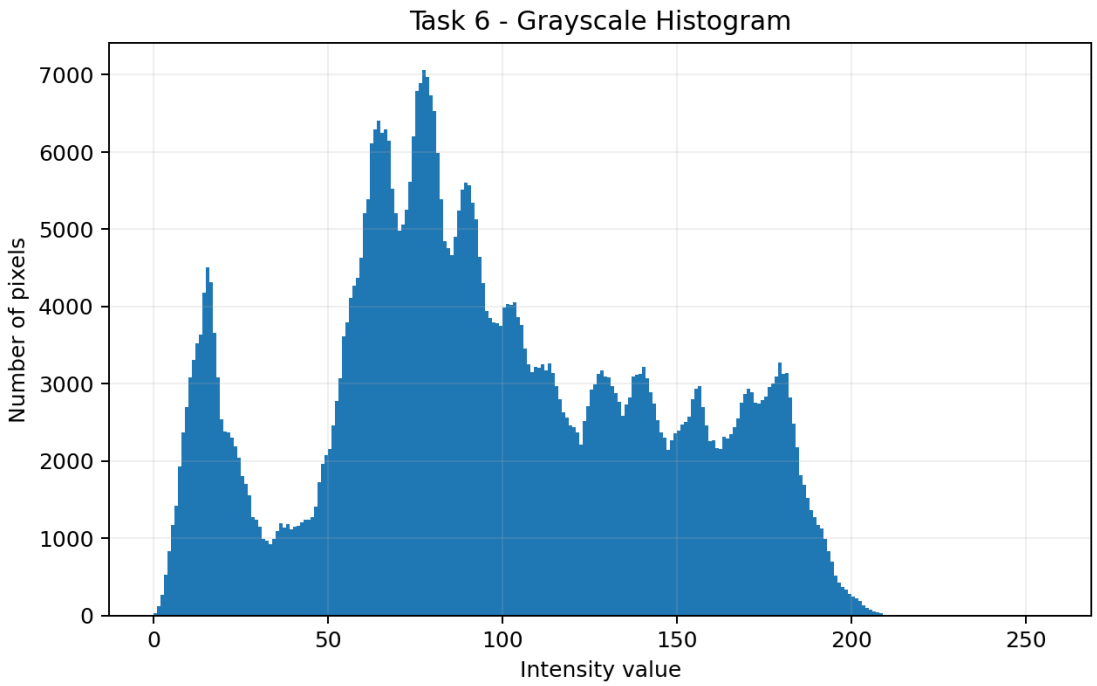


Figure 6. Histogram of the selected grayscale chest X-ray.

7.2 RGB histograms

The RGB image contains a large black background, which creates a strong peak near zero in all three channel histograms. Beyond the background, the green channel occupies high intensities more frequently than the other channels. This is consistent with the visual image, where green/yellow regions cover a large part of the active brain area. The red channel has a smaller but important high-intensity component, while the blue channel mainly represents cooler boundary structures.

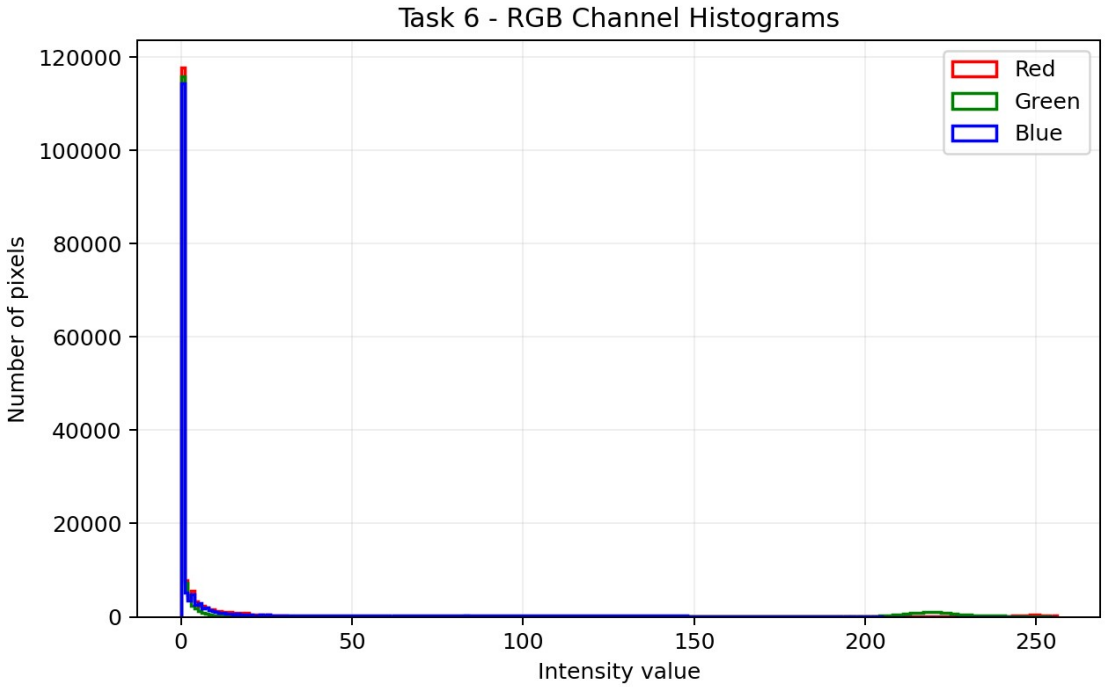


Figure 7. Individual Red, Green and Blue histograms of the pseudo-color biomedical image.

Histograms are useful for quickly judging exposure and contrast. A distribution crowded near zero suggests a dark image, a distribution crowded near 255 suggests a bright image, and a narrow distribution generally indicates low contrast. A wide distribution can indicate higher global contrast, although spatial information is not represented in a histogram.

7.3 MATLAB code used for Task 6

```
figure;
imhist(grayImg, 256);

R = rgbImg(:,:,1); G = rgbImg(:,:,2); B = rgbImg(:,:,3);
[hR,x] = imhist(R,256);
[hG,~] = imhist(G,256);
[hB,~] = imhist(B,256);
plot(x,hR,'r'); hold on;
plot(x,hG,'g');
plot(x,hB,'b');
```

8. Task 7 - Histogram Equalization

Global histogram equalization remaps intensity values using the cumulative histogram so that the available output range is used more evenly. For this experiment I used 256 output levels to match the 8-bit image range.

$$\textit{Equalization mapping: } s = (L - 1) \cdot CDF(r), \quad \textit{with } L = 256$$

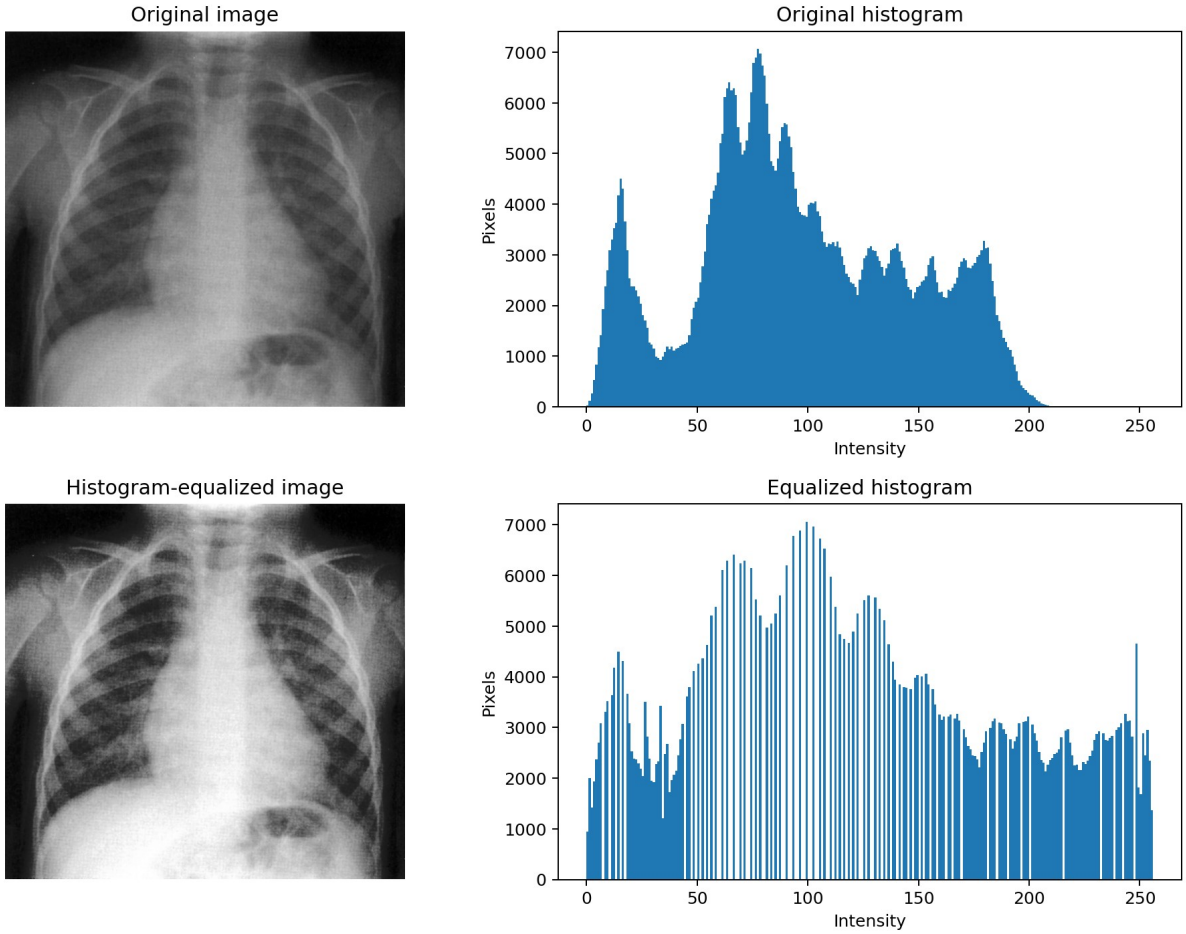


Figure 8. Original/equalized chest X-ray and the corresponding histograms.

After equalization, the output spans 0-255 and the standard deviation increases from 48.65 to 73.48. This wider spread makes local differences in the lungs, ribs and surrounding tissue more visible. The histogram

is not perfectly flat because the input has discrete repeated values and the mapping is one-to-one for groups of input gray levels, but it is distributed across a much larger portion of the range.

Histogram equalization does not always give a visually better result. It is a global operation, so it can over-emphasize noise or make some regions look unnatural. If the useful information is limited to a local area, adaptive methods such as CLAHE can be more suitable, although adaptive equalization was not required in this assignment.

8.1 MATLAB code used for Task 7

```
equalizedImg = histeq(grayImg, 256);

figure;
tiledlayout(2,2);
nexttile; imshow(grayImg, []);       title('Original image');
nexttile; imhist(grayImg, 256);      title('Original histogram');
nexttile; imshow(equalizedImg, []);  title('Histogram-equalized image');
nexttile; imhist(equalizedImg, 256); title('Equalized histogram');
```

9. Task 8 - Image Enhancement Comparison

For the point-operation enhancement, I used gain 1.25 and a brightness offset of +15. This makes the X-ray brighter and increases the intensity spread while keeping the transformation simple and predictable. I then compared it with the global histogram-equalized result from Task 7.

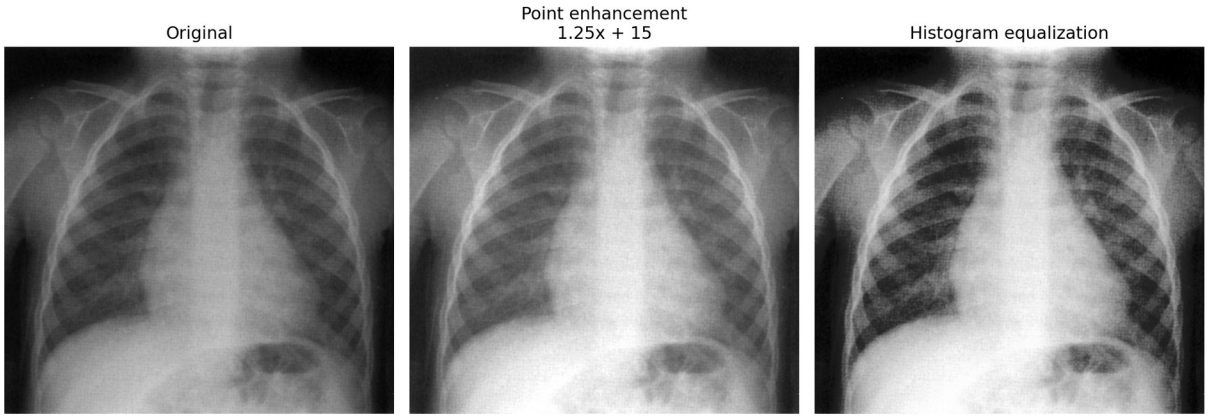


Figure 9. Side-by-side comparison of original, point-enhanced and histogram-equalized images.

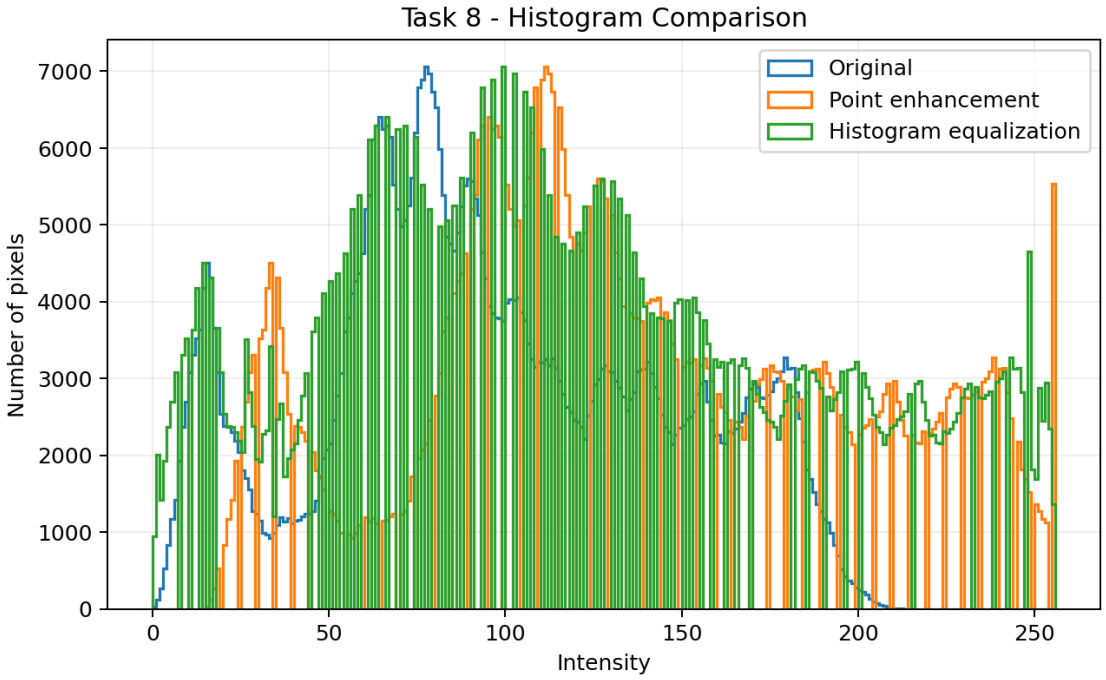


Figure 10. Histogram comparison for the three images.

Image	Min	Max	Mean	Std. dev.	Entropy (bits)
Original	0	212	96.247	48.646	7.4500
Point enhancement	15	255	134.890	60.718	7.4168
Histogram equalization	0	255	128.346	73.477	7.3581

Table 4. Quantitative comparison of the enhancement methods.

9.1 Which method was better for this image?

For the selected chest X-ray, histogram equalization gives the stronger global enhancement. Its standard deviation is 73.48 compared with 60.72 for the point-enhanced image, and its histogram uses the full 0-255 range. Visually, details in the lung fields and bony structures are separated more clearly. The point operation is still useful because it gives controlled brightness/contrast changes and looks less aggressive. Therefore, I would choose histogram equalization when the goal is to reveal more global detail in this particular image, but I would prefer the point operation when a more conservative appearance is required.

The entropy value does not increase after equalization in this example. This is not a contradiction: histogram equalization is designed to redistribute gray levels and improve contrast, not necessarily to maximize entropy for every discrete image. The visual result and the increased intensity spread are more directly relevant to the required comparison.

9.2 MATLAB code used for Task 8

```
pointEnhanced = clipUint8(1.25 * double(grayImg) + 15);
equalizedImg = histeq(grayImg, 256);

figure;
tiledlayout(1,3);
nexttile; imshow(grayImg,[]);       title('Original');
nexttile; imshow(pointEnhanced,[]); title('Point: 1.25x + 15');
nexttile; imshow(equalizedImg,[]);  title('Histogram equalization');
```

10. Implementation Challenges and Notes

- Some grayscale biomedical samples are stored as three-channel JPEG files. I therefore checked the number of channels before using rgb2gray.
- Arithmetic on uint8 values can hide clipping behavior. I converted to double for brightness/contrast calculations and explicitly clipped the output to 0-255 before converting back to uint8.
- DICOM data uses a different numerical range from normal 8-bit images. The selected file allocates 16 bits and stores 12 meaningful bits, so display scaling/windowing is required.
- A histogram describes intensity frequency but not spatial position. For that reason, histogram-based conclusions were always checked against the displayed image.
- Histogram equalization can increase global contrast but can also make noise or already-high-contrast areas too strong. It should not be treated as universally better.

The code uses relative paths based on the location of main.m. This keeps the repository portable: after cloning or extracting the project, the scripts can find data/ and outputs/ without changing a Windows drive letter.

11. Conclusion

This assignment showed how image processing operations can be understood directly in terms of matrices and intensity distributions. The grayscale image was represented as an M x N matrix, the RGB image as three separate M x N planes, and the DICOM file as a medical image object that also contains acquisition metadata. Point operations changed brightness and contrast in a simple pixel-by-pixel manner, while histogram analysis provided a global description of the intensity distribution.

For the selected chest X-ray, global histogram equalization produced the strongest overall contrast improvement because it spread the intensity values over the full 8-bit range. However, the comparison also showed that enhancement is application-dependent: stronger contrast is not automatically the most natural or clinically useful appearance. The main outcome of the assignment is therefore not only learning the MATLAB functions, but also learning how to interpret what each transformation does to the underlying pixel data.

Appendix A - Complete MATLAB Source Code

The following files are the complete source code included in the repository. The main script runs Tasks 1-8 and calls the helper functions below.

main.m

```
%% EE3005 Image Processing - Assignment 1
% Fundamentals of Image Representation and Enhancement
% Student: Burak Bekir Cakirer
% Student ID: 2211011008
%
% Run this script from MATLAB. It reads the supplied images from ../data,
% performs all assignment tasks, and writes figures to ../outputs.

clear; close all; clc;

srcDir = fileparts(mfilename('fullpath'));
addpath(srcDir);
rootDir = fileparts(srcDir);
dataDir = fullfile(rootDir, 'data');
outDir  = fullfile(rootDir, 'outputs');
if ~exist(outDir, 'dir'), mkdir(outDir); end

%% Task 1 - Image reading and matrix representation
img1 = imread(fullfile(dataDir, 'image3.jpg'));
if size(img1,3) == 3
    grayImg = rgb2gray(img1);
else
    grayImg = img1;
end

fprintf('Task 1: image size = %d x %d\n', size(grayImg,1), size(grayImg,2));
displayMatrixRepresentation(grayImg, outDir);

f = figure('Name','Task 1 - Grayscale Image');
imshow(grayImg, []);
title(sprintf('Grayscale Biomedical Image (%d x %d)', size(grayImg,1), size(grayImg,2)));
exportgraphics(f, fullfile(outDir,'matlab_task1_grayscale_image.png'), 'Resolution', 180);

%% Task 2 - Separating RGB channels
rgbImg = imread(fullfile(dataDir, 'image_rgb_matlab.png'));
[R,G,B] = separateRGBChannels(rgbImg, outDir);

%% Task 3 - Summary and applications
% This task is mainly discussed in the report. In MATLAB, the important
% concept is that grayscale images are MxN matrices and RGB images are
% MxNx3 arrays. Each channel can therefore be processed independently.

%% Task 4 - DICOM reading, display, and metadata extraction
dicomPath = fullfile(dataDir, 'image4.dcm');
dicomImg = dicomread(dicomPath);
dicomMeta = dicominfo(dicomPath);

f = figure('Name','Task 4 - DICOM');
% DICOM images may contain 12-bit or 16-bit data. imshow(...,[]) scales the
% stored range for display without changing the original pixel matrix.
imshow(dicomImg, []);
title('DICOM MR Image');
exportgraphics(f, fullfile(outDir,'matlab_task4_dicom_image.png'), 'Resolution', 180);

displayDicomMetadata(dicomMeta);

%% Task 5 - Point operations
pointResults = pointOperations(grayImg, outDir);

%% Task 6 - Histogram analysis
f = figure('Name','Task 6 - Grayscale Histogram');
imhist(grayImg, 256);
title('Grayscale Histogram');
xlabel('Intensity value'); ylabel('Number of pixels');
exportgraphics(f, fullfile(outDir,'matlab_task6_grayscale_histogram.png'), 'Resolution', 180);

plotRGBHistograms(rgbImg, outDir);

%% Task 7 - Histogram equalization
% Using 256 bins makes the mapping explicitly cover the full 8-bit range.
equalizedImg = histeq(grayImg, 256);

f = figure('Name','Task 7 - Histogram Equalization');
tiledlayout(2,2,'Padding','compact','TileSpacing','compact');
nexttile; imshow(grayImg, []); title('Original image');
nexttile; imhist(grayImg, 256); title('Original histogram');
nexttile; imshow(equalizedImg, []); title('Histogram-equalized image');
nexttile; imhist(equalizedImg, 256); title('Equalized histogram');
exportgraphics(f, fullfile(outDir,'matlab_task7_histogram_equalization.png'), 'Resolution', 180);

%% Task 8 - Image enhancement comparison
% Point enhancement: gain followed by brightness offset.
pointEnhanced = clipUint8(1.25 * double(grayImg) + 15);

f = figure('Name','Task 8 - Enhancement Comparison');
```

```
tiledlayout(1,3,'Padding','compact','TileSpacing','compact');
nexttile; imshow(grayImg,[]); title('Original');
nexttile; imshow(pointEnhanced,[]); title('Point: 1.25x + 15');
nexttile; imshow(equalizedImg,[]); title('Histogram equalization');
exportgraphics(f, fullfile(outDir,'matlab_task8_enhancement_comparison.png'), 'Resolution', 180);

f = figure('Name','Task 8 - Histogram Comparison');
[h0,x0] = imhist(grayImg,256);
[hp,xp] = imhist(pointEnhanced,256);
[he,xe] = imhist(equalizedImg,256);
plot(x0,h0,'LineWidth',1.0); hold on;
plot(xp,hp,'LineWidth',1.0);
plot(xe,he,'LineWidth',1.0);
grid on; xlim([0 255]);
xlabel('Intensity value'); ylabel('Number of pixels');
title('Histogram Comparison');
legend('Original','Point enhancement','Histogram equalization','Location','best');
exportgraphics(f, fullfile(outDir,'matlab_task8_histogram_comparison.png'), 'Resolution', 180);

%% Numerical metrics used in the report
names = {'Original','Brightness +50','Brightness -50','Contrast x1.5', ...
         'Contrast x0.6','Negative','Point enhancement','Histogram equalization'};
imgs = {grayImg,pointResults.bright,pointResults.dark,pointResults.highContrast, ...
        pointResults.lowContrast,pointResults.negative,pointEnhanced,equalizedImg};

fprintf('\nImage metrics (8-bit scale):\n');
fprintf('%-24s %8s %8s %10s %10s\n','Image','Min','Max','Mean','StdDev');
for k = 1:numel(imgs)
    a = double(imgs{k});
    fprintf('%-24s %8.0f %8.0f %10.3f %10.3f\n', names{k}, min(a(:)), max(a(:)), mean(a(:)), std(a(:),1));
end

fprintf('\nAll tasks finished. Outputs saved to:\n%s\n', outDir);
```

displayMatrixRepresentation.m

```
function displayMatrixRepresentation(I, outDir)
%DISPLAYMATRIXREPRESENTATION Display an image as numerical matrix values.
%   I is expected to be a 2-D grayscale matrix. A 10x10 region from the
%   image center is printed and displayed with intensity values.

if ndims(I) ~= 2
    error('Input must be a 2-D grayscale image.');
end

[nRows,nCols] = size(I);
r0 = floor(nRows/2)-4;
c0 = floor(nCols/2)-4;
block = I(r0:r0+9, c0:c0+9);

fprintf('\n10x10 sample from the center of the image:\n');
disp(block);

f = figure('Name','Task 1 - Matrix Representation');
imagesc(block); axis image; colormap gray; colorbar;
title('10 x 10 Matrix Sample from Image Center');
xlabel('Column index within crop'); ylabel('Row index within crop');

for r = 1:size(block,1)
    for c = 1:size(block,2)
        if block(r,c) < 110
            textColor = 'w';
        else
            textColor = 'k';
        end
        text(c,r,num2str(block(r,c)), 'HorizontalAlignment','center', ...
            'FontSize',8,'Color',textColor);
    end
end

if nargin > 1 && ~isempty(outDir)
    exportgraphics(f, fullfile(outDir,'matlab_task1_matrix_sample.png'), 'Resolution', 180);
    writematrix(block, fullfile(outDir,'matlab_task1_matrix_sample.csv'));
end
end
```

separateRGBChannels.m

```
function [R,G,B] = separateRGBChannels(rgbImg, outDir)
%SEPARATERGBCHANNELS Separate an MxNx3 RGB image into three MxN matrices.

if ndims(rgbImg) ~= 3 || size(rgbImg,3) ~= 3
    error('Input must be an RGB image with size M x N x 3.');
end

R = rgbImg(:,:,1);
G = rgbImg(:,:,2);
B = rgbImg(:,:,3);

f = figure('Name','Task 2 - RGB Channels');
```

```
tiledlayout(2,2,'Padding','compact','TileSpacing','compact');
nexttile; imshow(rgbImg); title('Original RGB image');
nexttile; imshow(R); title('Red channel');
nexttile; imshow(G); title('Green channel');
nexttile; imshow(B); title('Blue channel');

if nargin > 1 && ~isempty(outDir)
    exportgraphics(f, fullfile(outDir,'matlab_task2_rgb_channels.png'), 'Resolution', 180);
end
end
```

displayDicomMetadata.m

```
function displayDicomMetadata(info)
%DISPLAYDICOMMETADATA Print selected DICOM metadata fields when available.

fields = { ...
    'PatientName','PatientID','PatientBirthDate','PatientSex','PatientAge', ...
    'Modality','Manufacturer','ManufacturerModelName','StudyDate', ...
    'SeriesDescription','SliceThickness','RepetitionTime','EchoTime', ...
    'MagneticFieldStrength','FlipAngle','PhotometricInterpretation', ...
    'Rows','Columns','BitsAllocated','WindowCenter','WindowWidth'};

fprintf('\nSelected DICOM metadata:\n');
for k = 1:numel(fields)
    fieldName = fields{k};
    if isfield(info,fieldName)
        valueText = valueToText(info.(fieldName));
        fprintf('%-28s : %s\n', fieldName, valueText);
    end
end
end

function txt = valueToText(value)
% Convert common dicominfo field types to readable text.
if isstruct(value)
    names = fieldnames(value);
    parts = strings(1,numel(names));
    for n = 1:numel(names)
        v = value.(names{n});
        if isnumeric(v)
            v = mat2str(v);
        elseif isstring(v)
            v = char(v);
        end
        if ischar(v)
            parts(n) = string(v);
        else
            parts(n) = string(names{n});
        end
    end
    txt = char(strjoin(parts(parts ~= ""), ' '));
elseif isnumeric(value)
    txt = mat2str(value);
elseif isstring(value)
    txt = char(value);
elseif ischar(value)
    txt = value;
else
    txt = '<unhandled value type>';
end
end
```

pointOperations.m

```
function results = pointOperations(I, outDir)
%POINTOPERATIONS Brightness, contrast and negative point transformations.

if ~isa(I,'uint8')
    I = im2uint8(I);
end

results.bright = clipUint8(double(I) + 50);
results.dark = clipUint8(double(I) - 50);
results.highContrast = clipUint8(1.5 * double(I));
results.lowContrast = clipUint8(0.6 * double(I));
results.negative = 255 - I;

f = figure('Name','Task 5 - Point Operations');
tiledlayout(2,3,'Padding','compact','TileSpacing','compact');
nexttile; imshow(I); title('Original');
nexttile; imshow(results.bright); title('Brightness +50');
nexttile; imshow(results.dark); title('Brightness -50');
nexttile; imshow(results.highContrast); title('Contrast gain = 1.5');
nexttile; imshow(results.lowContrast); title('Contrast gain = 0.6');
nexttile; imshow(results.negative); title('Negative');

if nargin > 1 && ~isempty(outDir)
    exportgraphics(f, fullfile(outDir,'matlab_task5_point_operations.png'), 'Resolution', 180);
end
```

```
end
```

plotRGBHistograms.m

```
function plotRGBHistograms(rgbImg, outDir)
%PLOTRGBHISTOGRAMS Plot the histogram of each RGB channel.

R = rgbImg(:,:,1); G = rgbImg(:,:,2); B = rgbImg(:,:,3);
[hR,x] = imhist(R,256);
[hG,~] = imhist(G,256);
[hB,~] = imhist(B,256);

f = figure('Name','Task 6 - RGB Histograms');
plot(x,hR,'r','LineWidth',1.1); hold on;
plot(x,hG,'g','LineWidth',1.1);
plot(x,hB,'b','LineWidth',1.1);
grid on; xlim([0 255]);
xlabel('Intensity value'); ylabel('Number of pixels');
title('RGB Channel Histograms');
legend('Red','Green','Blue','Location','best');

if nargin > 1 && ~isempty(outDir)
    exportgraphics(f, fullfile(outDir,'matlab_task6_rgb_histograms.png'), 'Resolution', 180);
end
end
```

clipUint8.m

```
function out = clipUint8(x)
%CLIPUINT8 Clip a numerical array to [0,255] and convert it to uint8.
out = uint8(min(max(round(x),0),255));
end
```

Appendix B - MATLAB Functions Used

Core MATLAB/Image Processing Toolbox functions used in the implementation: imread, rgb2gray, imshow, imagesc, dicomread, dicominfo, imhist, histeq, tiledlayout, exportgraphics, and writematrix.